

MUST University

Department of Computer Science

CS 141 – Data Structures & Algorithms

Spring 2026

1st Year – Software Engineering License

CHAPTER SERIES

Chapter 3 – Linked Lists

Singly Linked Lists: Concepts, Operations & Exercises

Topics Covered

- Introduction to Linked Lists
- Node Structure (data + pointer)
 - List Traversal & Display
- Insertion: Beginning – End – Middle
- Deletion: By Position & By Value
- Applied Exercises: Integer List & Student Records

Prepared by the CS Department • Spring 2026

Table of Contents

1. Introduction to Linked Lists

A **linked list** is a fundamental linear data structure in which elements — called **nodes** — are stored at non-contiguous memory locations and connected through **pointers**. Unlike arrays, linked lists do not require elements to be stored in adjacent memory cells, which gives them great flexibility for dynamic memory management.

Each node in a singly linked list holds two pieces of information:

- **data** — the actual value stored in the node (e.g., an integer, a string, a struct).
- **next** — a pointer to the next node in the list (or NULL if it is the last node).

1.1 Arrays vs. Linked Lists

Feature	Array	Linked List	Notes
Memory	Contiguous	Non-contiguous	LL is more flexible
Size	Fixed (static)	Dynamic	LL grows/shrinks freely
Access	O(1) random	O(n) sequential	Array faster for read
Insertion	O(n)	O(1) at head	LL faster at beginning
Deletion	O(n)	O(1) at head	LL faster at beginning
Memory overhead	None	+pointer per node	Small overhead per node

1.2 Visual Representation

A singly linked list with three integer nodes ($10 \rightarrow 20 \rightarrow 30$) is shown below. Each box has two compartments: the left holds the data and the right holds the pointer to the next node.

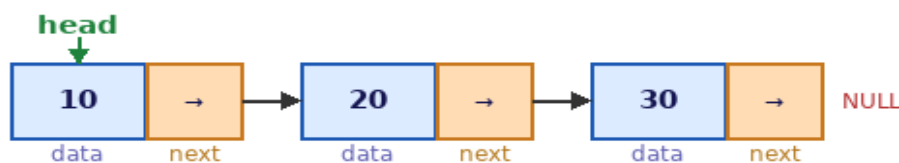


Figure 1.1 – Singly Linked List: $10 \rightarrow 20 \rightarrow 30 \rightarrow \text{NULL}$

The **head** pointer always references the first node. The last node's **next** field is set to **NULL** to signal the end of the list.

2. Node Structure & Syntax

2.1 Defining a Node in C

In C, a node is defined using a **struct**. The data field stores an integer and the next field is a self-referential pointer — it points to another node of the same type.

```
// — Node definition —
struct Node {
    int data;           // The integer value stored
    struct Node* next;  // Pointer to the next node
};

// Convenience typedef
typedef struct Node Node;
```

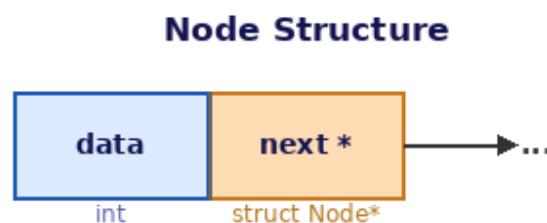


Figure 2.1 – Node structure: data (int) | next (pointer to Node)

2.2 Creating a Node Dynamically

Nodes are created on the **heap** using `malloc()`. Always check that the allocation succeeded before using the pointer.

```
#include <stdio.h>
#include <stdlib.h>

// Allocate and initialise a new node
Node* createNode(int value) {
    Node* newNode = (Node*) malloc(sizeof(Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return NULL;
    }
    newNode->data = value;    // Store the integer
    newNode->next = NULL;    // No successor yet
    return newNode;
}
```

Key Point – Dynamic vs. Static Allocation

`malloc()` allocates memory at runtime on the heap.

The size of the list can change during program execution.

Always call `free()` on every node when the list is no longer needed to avoid memory leaks.

2.3 Traversing & Printing the List

To visit every node we use a **temporary pointer** (often called `current` or `temp`) that walks along the list until it reaches `NULL`.

```
// Print all integers in the list
void printList(Node* head) {
    Node* current = head;      // Start from the head
    while (current != NULL) {
        printf("%d -> ", current->data);
        current = current->next; // Move to next node
    }
    printf("NULL\n");
}
```

3. Insertion Operations

A linked list supports three primary insertion positions: at the **beginning**, at the **end**, and in the **middle** (at a given position). Each case requires different pointer manipulation.

3.1 Insert at the Beginning

Inserting at the beginning is the **most efficient** operation — $O(1)$ — because no traversal is needed. The new node simply becomes the new head.

Algorithm: (1) Allocate new node. (2) Set `newNode→next = head`. (3) Update `head = newNode`.

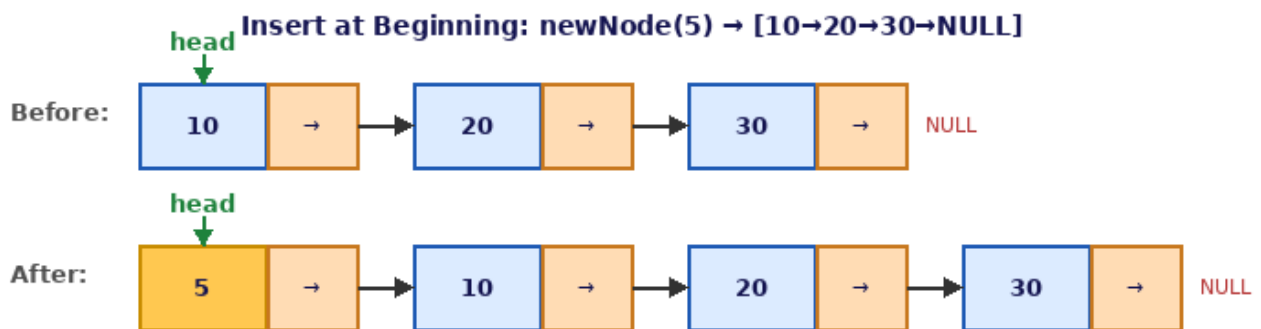


Figure 3.1 – Inserting node(5) at the beginning of [10 → 20 → 30]

```
// Insert a new node at the BEGINNING of the list
Node* insertAtBeginning(Node* head, int value) {
    Node* newNode = createNode(value);
    if (newNode == NULL) return head; // Allocation failed

    newNode->next = head; // New node points to old head
    head = newNode;      // Head now points to new node
    return head;
}

// Usage example
int main() {
    Node* head = NULL; // Empty list
    head = insertAtBeginning(head, 30);
    head = insertAtBeginning(head, 20);
    head = insertAtBeginning(head, 10);
    printList(head); // Output: 10 -> 20 -> 30 -> NULL
    return 0;
}
```

3.2 Insert at the End

To insert at the end, we must **traverse the entire list** to find the last node (the one whose `next` is NULL), then link it to the new node. Time complexity: $O(n)$.

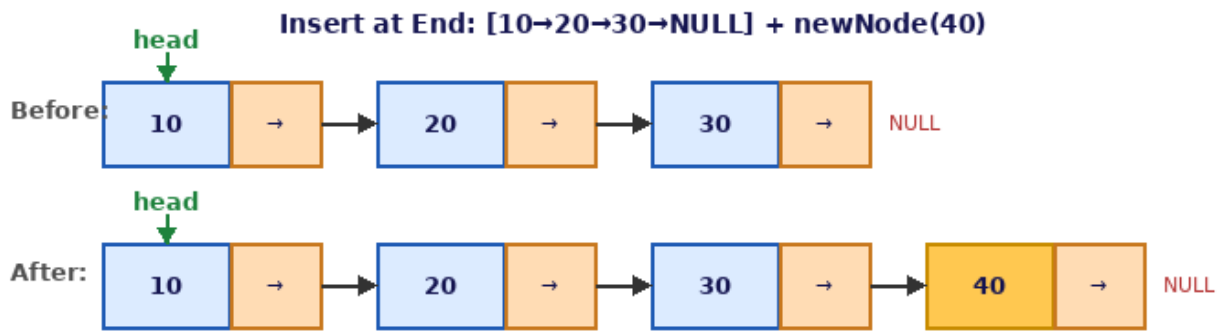


Figure 3.2 – Inserting node(40) at the end of [10 → 20 → 30]

```
// Insert a new node at the END of the list
Node* insertAtEnd(Node* head, int value) {
    Node* newNode = createNode(value);
    if (newNode == NULL) return head;

    if (head == NULL) return newNode; // List was empty

    Node* current = head;
    while (current->next != NULL) { // Walk to last node
        current = current->next;
    }
    current->next = newNode; // Link last node to new node
    return head;
}
```

3.3 Insert at a Given Position (Middle)

Inserting at position **pos** (1-indexed) requires traversing to node `pos - 1`, then relinking pointers. If `pos = 1` it degenerates to `insertAtBeginning`. Time complexity: $O(n)$.

Algorithm: (1) Reach the node just before the target position. (2) Set `newNode→next = prev→next`. (3) Set `prev→next = newNode`.

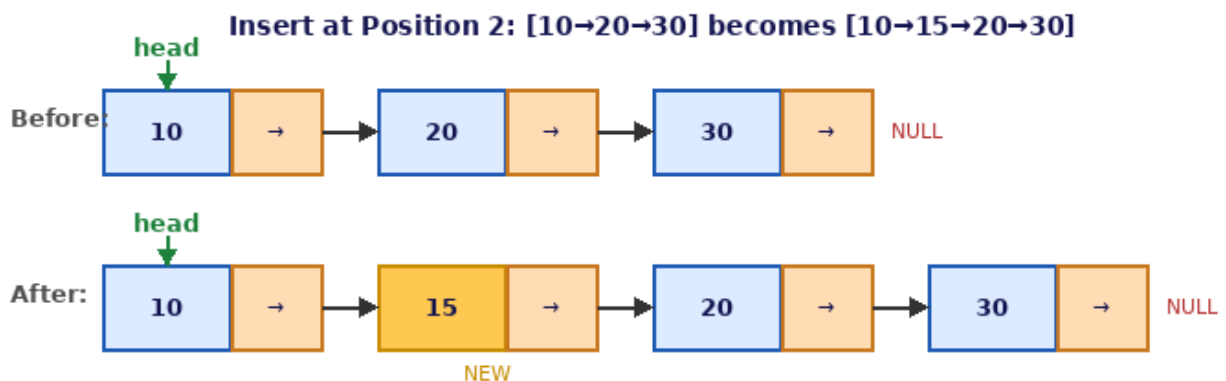


Figure 3.3 – Inserting node(15) at position 2 in [10 → 20 → 30]

```
// Insert at position pos (1-indexed)
Node* insertAtPosition(Node* head, int value, int pos) {
    if (pos <= 1) return insertAtBeginning(head, value);

    Node* newNode = createNode(value);
    if (newNode == NULL) return head;

    Node* current = head;
    int i = 1;
    while (current != NULL && i < pos - 1) {
        current = current->next;
        i++;
    }
    if (current == NULL) {
        printf("Position out of range.\n");
        free(newNode);
        return head;
    }
    newNode->next = current->next; // Link new node forward
    current->next = newNode;      // Link predecessor to new
    return head;
}
```

3.4 Complete Integer List Program

Below is a complete, runnable C program demonstrating all three insertion operations:

```
#include <stdio.h>
#include <stdlib.h>

struct Node { int data; struct Node* next; };
typedef struct Node Node;

Node* createNode(int v) {
    Node* n = (Node*)malloc(sizeof(Node));
    n->data = v; n->next = NULL; return n;
}

void printList(Node* head) {
    while (head) { printf("%d -> ", head->data); head=head->next; }
    printf("NULL\n");
}

Node* insertAtBeginning(Node* head, int v) {
    Node* n = createNode(v); n->next = head; return n;
}

Node* insertAtEnd(Node* head, int v) {
    Node* n = createNode(v);
    if (!head) return n;
    Node* c = head;
    while (c->next) c = c->next;
    c->next = n; return head;
}

Node* insertAtPosition(Node* head, int v, int pos) {
    if (pos<=1) return insertAtBeginning(head,v);
}
```



```
Node* n=createNode(v); Node* c=head; int i=1;
while (c && i<pos-1) { c=c->next; i++; }
if (!c) { free(n); return head; }
n->next=c->next; c->next=n; return head;
}
int main() {
Node* head = NULL;
head = insertAtEnd(head, 10);
head = insertAtEnd(head, 30);
head = insertAtBeginning(head, 5);
head = insertAtPosition(head, 20, 3);
printList(head); // 5 -> 10 -> 20 -> 30 -> NULL
return 0;
}
```

Exercise 1 – Student Linked List

Objective

Apply the three insertion operations to a linked list of Student records.

Each student node must store: name (char[50]), gpa (float), age (int).

Step 1 – Define the Student Node

Begin by defining the struct that represents a student node:

```
// Student node structure
struct Student {
    char name[50];        // Full name of the student
    float gpa;            // Grade Point Average (0.0 - 4.0)
    int age;              // Age in years
    struct Student* next; // Pointer to next student
};
typedef struct Student Student;
```

Step 2 – Tasks

Task 2.1 – createStudent()

Write the function: Student* createStudent(char* name, float gpa, int age)

The function must allocate a new Student node, copy the name using strcpy(), assign the gpa and age, and return the new node.

Task 2.2 – printStudentList()

Write the function: void printStudentList(Student* head)

It should traverse the list and print each student's name, GPA, and age.

Example output: Alice (GPA: 3.8, Age: 20) -> Bob (GPA: 3.5, Age: 21) -> NULL

Task 2.3 – Insertion Functions

Implement the three functions:

- Student* insertStudentAtBeginning(Student* head, char* name, float gpa, int age)
- Student* insertStudentAtEnd(Student* head, char* name, float gpa, int age)
- Student* insertStudentAtPosition(Student* head, char* name, float gpa, int age, int pos)

Each mirrors the integer list logic but creates Student nodes.

Task 2.4 – Main Program

In `main()`, build a list of at least 5 students using a mix of the three functions.

Print the list after each insertion to verify correctness.

Sample students to use: Alice/3.8/20, Bob/3.5/21, Carol/3.9/19, David/2.7/22, Eva/3.2/20

Hints & Reminders

Include `<string.h>` to use `strcpy()` for copying the name string.

Use `%s` for name, `%.2f` for GPA, and `%d` for age in `printf()` format strings.

Remember to free all nodes with `free()` at the end of `main()`.

Test each insertion case independently before combining them.

4. Deletion Operations

Removing a node from a linked list requires careful pointer manipulation. We must **redirect the predecessor's next pointer** around the node to delete, then **free** its memory. Three cases arise: deleting from the beginning, from the end, and by value.

General Deletion Strategy

Step 1 – Find the node to delete (and keep track of its predecessor).
 Step 2 – Re-link the predecessor's next pointer to skip the target node.
 Step 3 – Call `free()` to release the node's memory back to the heap.
 Special case: if the target is the head, update head directly.

4.1 Delete from the Beginning

This is the simplest and fastest deletion — $O(1)$. Move head to the second node, then free the original first node.

```
// Delete the FIRST node of the list
Node* deleteFromBeginning(Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return NULL;
    }
    Node* temp = head;          // Save reference to old head
    head = head->next;          // Move head to second node
    free(temp);                 // Free old head node
    return head;
}
```

4.2 Delete from the End

We traverse to the **second-to-last** node, then set its `next` to `NULL`, freeing the last node. Time complexity: $O(n)$.

```
// Delete the LAST node of the list
Node* deleteFromEnd(Node* head) {
    if (head == NULL) return NULL;
    if (head->next == NULL) { free(head); return NULL; } // One node

    Node* current = head;
    while (current->next->next != NULL) { // Walk to 2nd-last
        current = current->next;
    }
    free(current->next); // Free the last node
    current->next = NULL; // Previous last → now the new end
    return head;
}
```

4.3 Delete by Value

This is the general case: find and remove the **first node** whose data matches the target value. Two sub-cases exist: the match is the head, or it is somewhere in the middle/end.

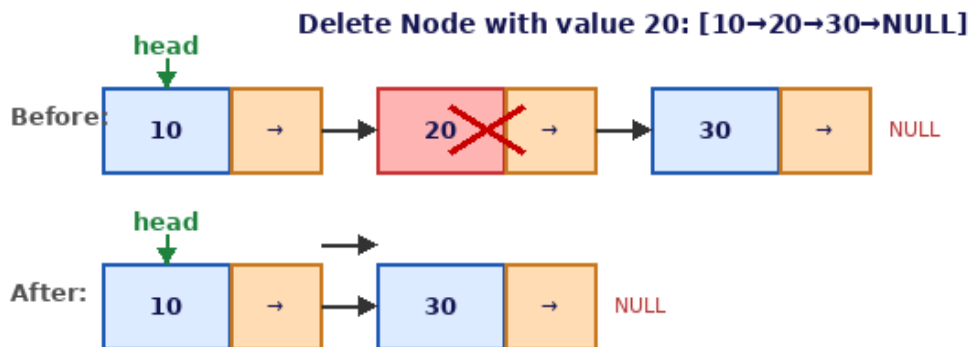


Figure 4.1 – Deleting node with value 20 from [10 → 20 → 30 → NULL]

```
// Delete the first node whose data == target
Node* deleteByValue(Node* head, int target) {
    if (head == NULL) return NULL; // Empty list

    // Case 1: target is the head node
    if (head->data == target) {
        Node* temp = head;
        head = head->next;
        free(temp);
        return head;
    }

    // Case 2: search inside the list
    Node* current = head;
    while (current->next != NULL) {
        if (current->next->data == target) {
            Node* toDelete = current->next;
            current->next = toDelete->next; // Skip the node
            free(toDelete);                // Release memory
            return head;
        }
        current = current->next;
    }
    printf("Value %d not found in the list.\n", target);
    return head;
}
```

4.4 Complete Integer List Program – All Deletions

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node { int data; struct Node* next; };
typedef struct Node Node;

Node* createNode(int v) {
    Node* n=(Node*)malloc(sizeof(Node)); n->data=v; n->next=NULL; return n;
}

Node* insertAtEnd(Node* h, int v) {
    Node* n=createNode(v);
    if(!h) return n;
    Node* c=h; while(c->next) c=c->next; c->next=n; return h;
}

void printList(Node* h) {
    while(h){printf("%d -> ",h->data);h=h->next;} printf("NULL\n");
}

Node* deleteFromBeginning(Node* h) {
    if(!h) return NULL;
    Node* t=h; h=h->next; free(t); return h;
}

Node* deleteFromEnd(Node* h) {
    if(!h) return NULL;
    if(!h->next){free(h); return NULL;}
    Node* c=h; while(c->next->next) c=c->next;
    free(c->next); c->next=NULL; return h;
}

Node* deleteByValue(Node* h, int target) {
    if(!h) return NULL;
    if(h->data==target){Node* t=h;h=h->next;free(t);return h;}
    Node* c=h;
    while(c->next){
        if(c->next->data==target){
            Node* t=c->next; c->next=t->next; free(t); return h;
        }
        c=c->next;
    }
    printf("%d not found.\n", target); return h;
}

int main() {
    Node* head=NULL;
    head=insertAtEnd(head,10); head=insertAtEnd(head,20);
    head=insertAtEnd(head,30); head=insertAtEnd(head,40);
    printf("Initial: "); printList(head);
    head=deleteFromBeginning(head);
    printf("After del head: "); printList(head);
    head=deleteByValue(head,30);
    printf("After del 30: "); printList(head);
    head=deleteFromEnd(head);
    printf("After del tail: "); printList(head);
    return 0;
}
```

Exercise 2 – Deleting Student Records

Objective

Extend your Student Linked List from Exercise 1 with deletion operations.
You must implement deletion by specific criteria (not just by position).

Step 1 – Delete Student by Name

Task 3.1 – deleteStudentByName()

Write: `Student* deleteStudentByName(Student* head, char* targetName)`
Remove the first student whose name exactly matches `targetName`.
Use `strcmp()` to compare names. Handle the case where the name is not found.
Test: build the list [Alice, Bob, Carol, David, Eva], then delete "Carol".
Expected: [Alice, Bob, David, Eva]

Step 2 – Delete Students Below GPA Threshold

Task 3.2 – deleteStudentsBelowGPA()

Write: `Student* deleteStudentsBelowGPA(Student* head, float threshold)`
Remove ALL students whose GPA is strictly less than `threshold`.
This requires iterating through the entire list (multiple deletions may occur).
Test: starting from [Alice/3.8, Bob/3.5, Carol/3.9, David/2.7, Eva/3.2],
call `deleteStudentsBelowGPA(head, 3.5)`.
Expected after deletion: [Alice/3.8, Bob/3.5, Carol/3.9]
Note: Bob (3.5) is kept because the condition is strictly LESS THAN 3.5.

Step 3 – Delete Oldest Student

Task 3.3 – deleteOldestStudent()

Write: `Student* deleteOldestStudent(Student* head)`
Find the student with the highest age and remove them from the list.
If multiple students share the same maximum age, remove the first one found.
Test with the original 5-student list and verify the correct node is removed.

Step 4 – Full Program

Task 3.4 – Complete main() with Memory Cleanup

Build the 5-student list, then perform all three deletion operations in order:

- (1) `deleteStudentByName(head, "Carol")`
- (2) `deleteStudentsBelowGPA(head, 3.5)` (on the result of step 1)
- (3) `deleteOldestStudent(head)` (on the result of step 2)

Print the list after each operation.

At the end, free all remaining nodes with a `freeList()` helper function.

Hints & Good Practices

Use a `prev` pointer alongside `current` when traversing for deletion.

Always check if `head == NULL` before any operation.

For `deleteStudentsBelowGPA()` use a loop that repeatedly checks the new head.

Write a helper `freeList(Student* head)` that frees every node — call it at end of `main()`.

Use `valgrind` (if available) to check for memory leaks: `valgrind ./program`

5. Chapter Summary

This chapter introduced the **Singly Linked List** — one of the most important data structures in computer science. Below is a concise reference for all operations covered:

Operation	Time	Space	Key Action
Insert at Beginning	$O(1)$	$O(1)$	Update head
Insert at End	$O(n)$	$O(1)$	Walk to last node
Insert at Position	$O(n)$	$O(1)$	Walk to pos-1
Delete from Beginning	$O(1)$	$O(1)$	Update head, free
Delete from End	$O(n)$	$O(1)$	Walk to 2nd-to-last
Delete by Value	$O(n)$	$O(1)$	Find predecessor, re-link
Traversal / Print	$O(n)$	$O(1)$	Follow next pointers

Key Takeaways

- A linked list stores elements in nodes, each containing data and a pointer to the next node.
- Insertion at the head is $O(1)$; all other insertions and deletions require $O(n)$ traversal.
- Always `free()` deleted nodes to avoid memory leaks in C.
- Linked lists are ideal when the number of elements is unknown at compile time.
- The same concepts apply to any data type stored in the node — int, float, struct, etc.